

Time Complexity

$$\underline{n^2} \quad \underline{n^3}$$

$\rightarrow n=10$
for (i=0 $\rightarrow$ n) {

}
for (i=0 $\rightarrow$ n) {
}
}

$O(n)$
 $\downarrow$
single

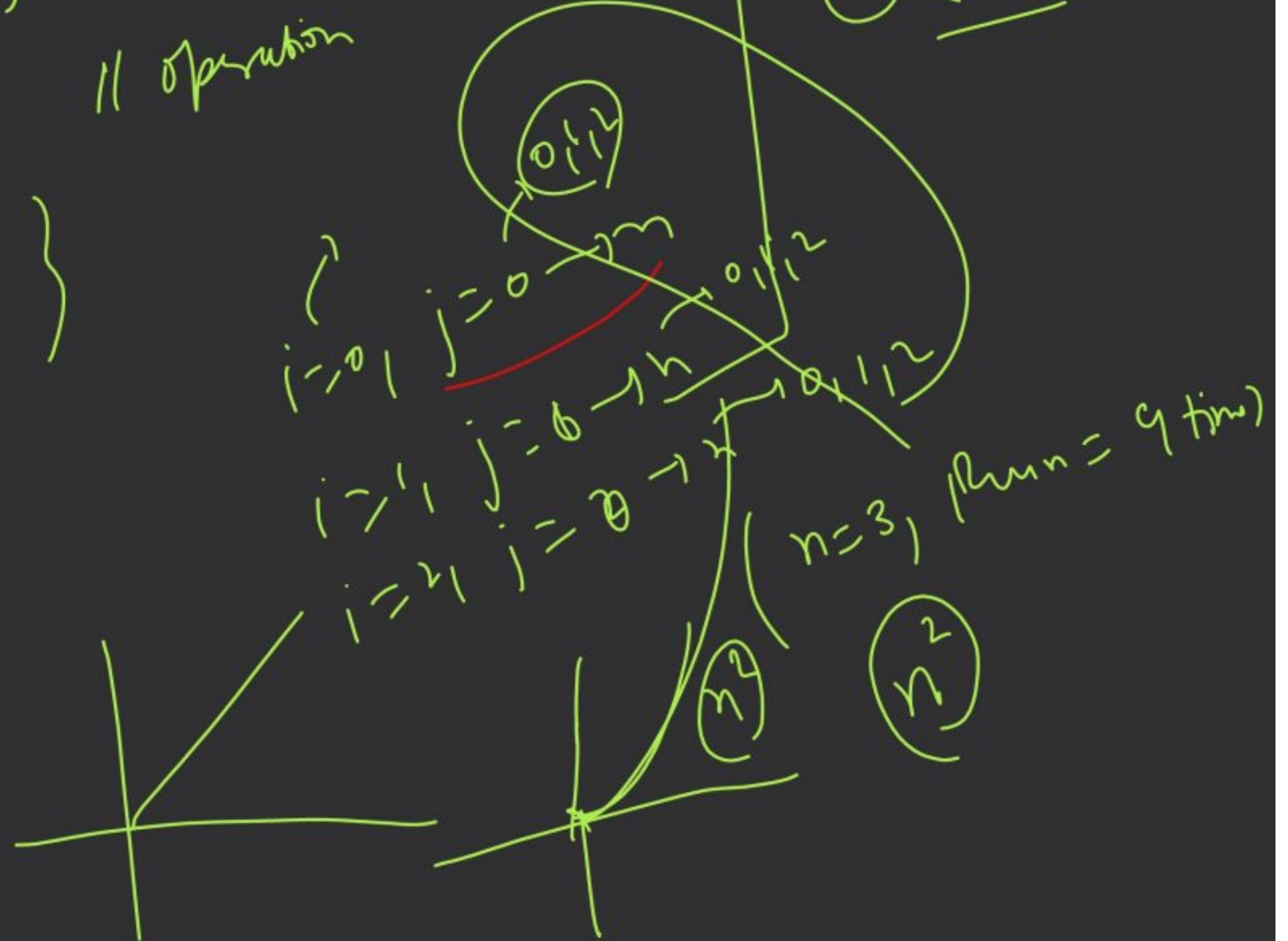
$$\underline{O(n)} + O(n) = \underline{2O(n)}$$

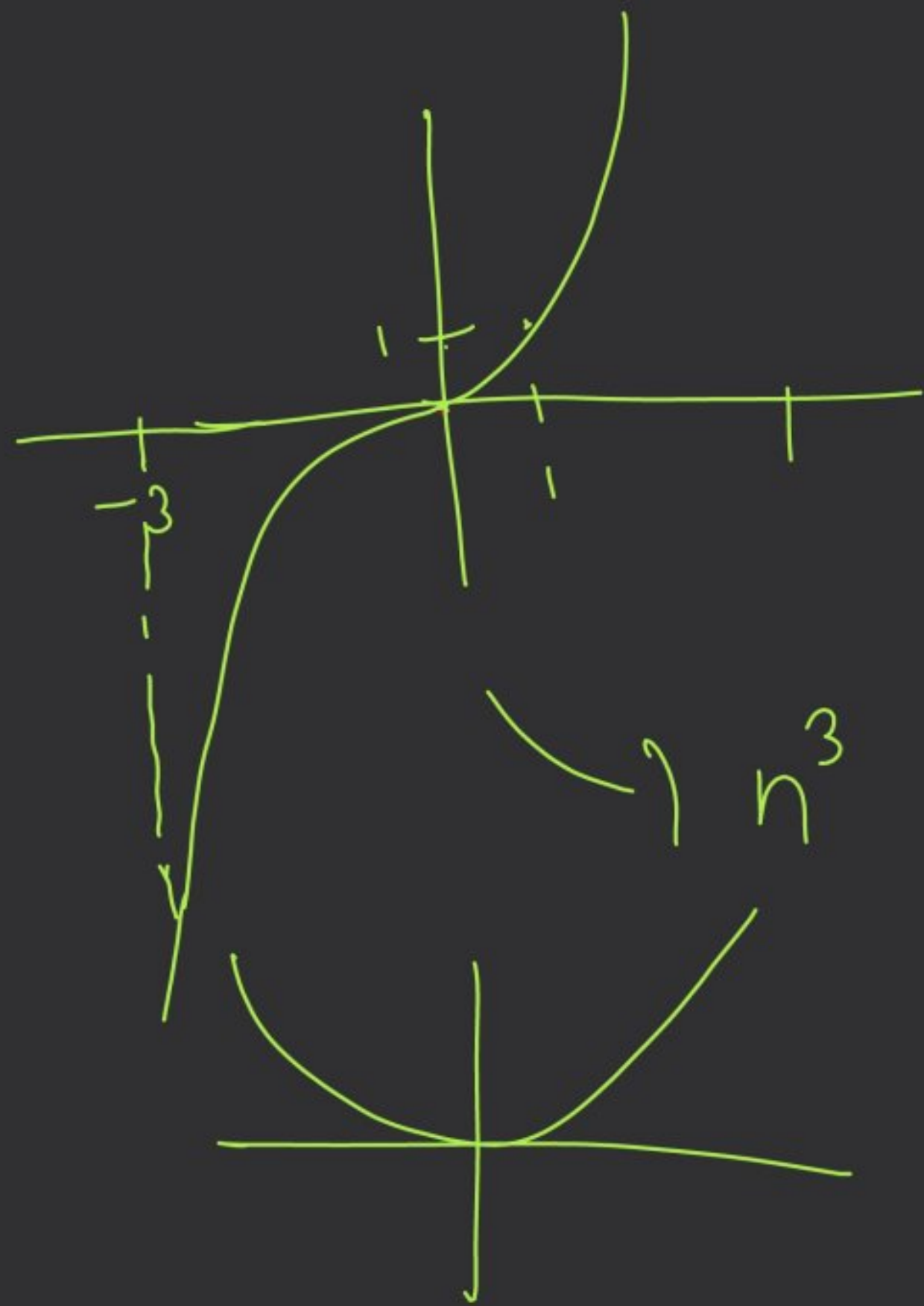
$\rightarrow 10$
 $= 17$

for (int i=0 $\rightarrow$ n) {
for (int j=0 $\rightarrow$ n) {

// operation

$O(n^2)$





$$\begin{array}{c}
 \rightarrow n^2 \quad n^3 \quad n^4 \quad n^5 \\
 \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\
 (-2)^2 \quad (-2)^3 \quad (-2)^4 \quad (-2)^5 \\
 \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\
 4 \quad 16 \quad 32 \quad 64
 \end{array}$$

Handwritten calculations for powers of -2 :

- $n^2 \rightarrow (-2)^2 = 4$
- $n^3 \rightarrow (-2)^3 = 16$
- $n^4 \rightarrow (-2)^4 = 32$
- $n^5 \rightarrow (-2)^5 = 64$

Binomial expansion of $(-2)^5$:

$$(-2)^5 = -2 \times -2 \times -2 \times -2 \times -2 = -32$$

Counted
↓
2
↓
(0)

Count Right
↓
3
↓
(1)

Count Left
↓
7
↓
(2)

$$O(n) + O(n) + O(n) + O(n)$$

$$+ O(4) \rightarrow 300 = \underline{1200}$$

```
for (int i=0; i < Count; i++) {
    nums[i] = 0;
}
```

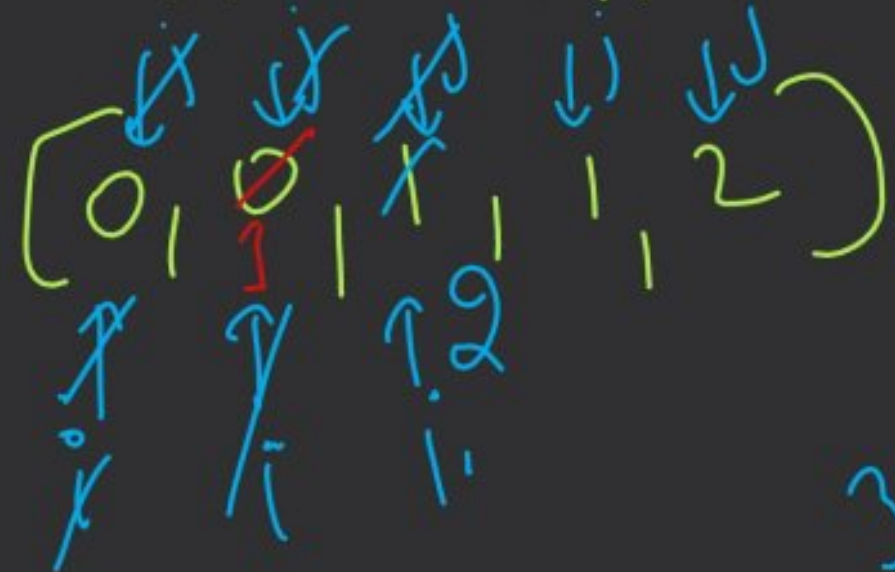
```
for (i=0; i < CountRight; i++) {
    nums[i] = 1;
}
```



→ Duplicates are always adjacent
 → Same element are grp together

→ Slow pointer (i) → unique element

→ Fast pointer (j) → Traverse the array



2

N

return i + 1

if same → duplicate → skip
 if different → new element → store it

int i = 0;

```

for (int j = 0; j < n; j++) {
    if (nums[j] != nums[i]) {

```

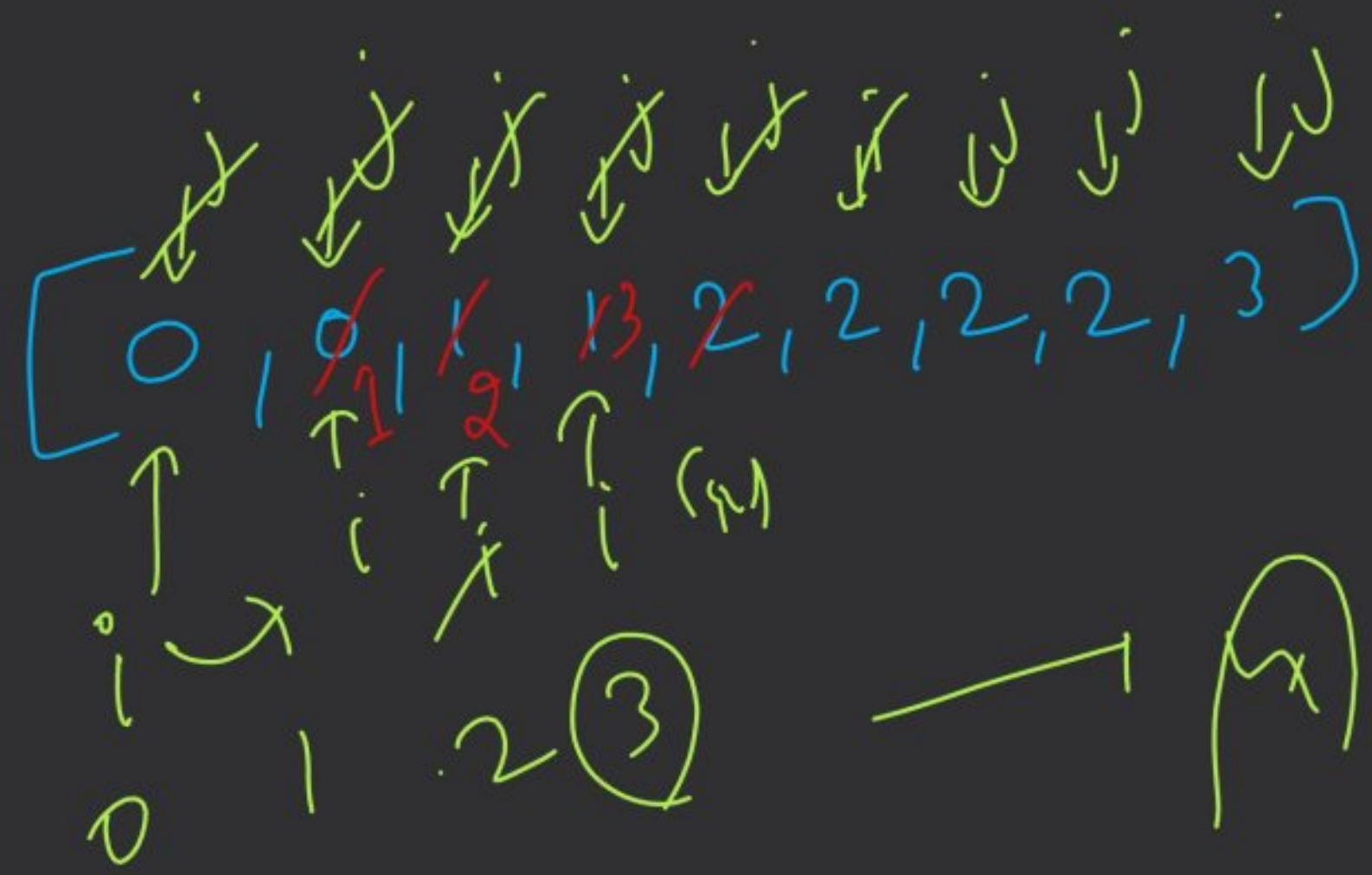
i++;

nums[i] = nums[j]

```

    }
}

```

$i = i - 1$

$num(i) = num(i, 1)$

$[0, 0, 0, 1, 1, 1, 2, 2, 3, 3, 3, 4, 5, 6, 7, 7]$

```
int i=0;
for (int j=0; j<n; j++) {
```

```
if (nums[j] != 0) {
```

// swap

```
i++;
```

```
}
```

```
nums(i), nums(j)
```

2 temp

```
int temp = nums(i);
```

```
nums(i) = nums(j);
```

```
nums(j) = temp;
```

```
i++;
```

Move Zeros:-

[0, 1, 0, 3, 12]

↔

[5, 4, 7, 8]

output [1, 3, 12, 0, 0]

[0, 0, 0, 3, 12]

[12, 8, 4, 0, 0, 0]

[12, 8, 4, 0, 0, 0]

Max^m Subarray ^{sum} $\downarrow$ $\downarrow$ $\downarrow$

Input = [-2, 1, 3, 4, -1, 2, 1, -5, 4]

Sum = sum + nums(i);

maxSum = Math.max(maxSum, sum);

(Continuous section of the array)

for (i = 0; i < n; i++)
sum = 0;
for (j =

$\begin{matrix} & & i & j & j \\ & & \downarrow & \downarrow & \downarrow \\ [-1, & 2, & 3, & 4, & 1] \end{matrix}$

maxSum = 0

for (i = 0 → n) {

sum = 0;

for (j = 0 → n) { j = 1

sum = sum + num[j];

maxSum = Math.max(sum, maxSum);

sum = 0 X 3 6 10 9

maxSum = 0 X 3 6 10

sum = 0 2 5 9

maxSum = 0 X 2 5 9

-100

[-2, -3, -1]

Integer.MAX_VALUE
 Integer.MIN_VALUE
 () → maxSum

[-2, 1]

-1

21
 -2
 201

-2
 -1
 -129

maxSum = Integer.MIN_VALUE
 sum = 0

[-2, 1, 2, 4, 0, -5, -100]

↑
(i=0)

(sum = -2)

maxSum = -2

[-2, 1]

$$\begin{matrix} j^i & i^i \\ [-2, +3, -5, -1, 1] \end{matrix}$$

$$Sum = -2 - 5 - 10 - 11 - 10$$

$$X \left(\maxSum = \min_val - 2, \right.$$

$$Sum = -2 \quad \emptyset \quad \times \quad -2 \quad \emptyset \quad \times \quad \emptyset$$

$$\maxSum = -2 \quad \}$$

$$\maxSum = \text{Integer.MIN_VALUE};$$

$$Sum = 0;$$

$$\text{for (int num : nums) \{}$$

$$Sum = Sum + num;$$

$$\maxSum = \text{Math.max}(\maxSum, Sum);$$

$$\text{if (Sum < 0) Sum = 0;}$$

$$\text{return maxSum;}$$



Problem List



1:1 Sessions with me



83



Premium

</> Code

Java Auto



```
7 for(int num : nums){
8     if(num == 0) countZero++;
9     if(num == 1) countOne++;
10    if(num == 2) countTwo++;
11 }
```

```
13 // countZero = 2 countOne = 3 countTwo = 4
```

```
15 int index = 0;
16 for(int i = 0; i < countZero; i++){
17     nums[index] = 0; // [0,0]
18     index++;
19 }
```

```
21 for(int i = 0; i < countOne; i++){
22     nums[index] = 1;
23     index++;
24 }
```

```
26 for(int i = 0; i < countTwo; i++){
27     nums[index] = 2;
28     index++;
29 }
```

*i = counting
index pointer is for copying the value in nums.
(0,0,1,1,1,2,2,2,2)*

In-place algorithm

14 languages

Contents hide

(Top)

Examples

In computational complexity

Role of randomness

In functional programming

See also

References

Article Talk

Read Edit View history Tools

From Wikipedia, the free encyclopedia

"In-place" redirects here. For execute in place file systems, see [Execute in place](#).



This article **needs additional citations for verification**. Please help [improve this article](#) by [adding citations to reliable sources](#). Unsourced material may be challenged and removed.
Find sources: "In-place algorithm" – news · newspapers · books · scholar · JSTOR (January 2015) (Learn how and when to remove this message)

In [computer science](#), an **in-place algorithm** is an [algorithm](#) that operates directly on the input [data structure](#) without requiring extra space proportional to the input size. In other words, it modifies the input in place, without creating a separate copy of the data structure. An algorithm which is not in-place is sometimes called **not-in-place** or **out-of-place**.

In-place can have slightly different meanings. In its strictest form, the algorithm can only have a [constant amount of extra space](#), counting everything including [function calls](#) and [pointers](#). However, this form is very limited as simply having an index to a length n array requires $O(\log n)$ bits. More broadly, in-place means that the algorithm does not use extra space for manipulating the input but may require a small though non-constant extra space for its operation. Usually, this space is $O(\log n)$, though sometimes anything in $o(n)$ is allowed. Note that space complexity also has varied choices in whether or not to count the index lengths as part of the space used. Often, the space complexity is given in terms of the number of indices or pointers needed, ignoring their length. In this article, we refer to total space complexity ([DSpace](#)), counting pointer lengths. Therefore, the space requirements here have an extra $\log n$ factor compared to an analysis that ignores the lengths of indices and pointers.

An algorithm may or may not count the output as part of its space usage. Since in-place algorithms usually overwrite their input with output, no additional space is needed. When writing the output to write-only memory or a stream, it may be more appropriate to only consider the working space of the algorithm. In theoretical applications such as [log-space reductions](#), it is more typical to always ignore output space (in these cases it is more essential that the output is *write-only*).

Appearance hide

Birthday mode (Baby Globe)

☐ Disabled
☒ Enabled

[Learn more about Birthday mode](#)

Text

☐ Small
☒ Standard
☐ Large

Width

☒ Standard
☐ Wide

Color (beta)

☐ Automatic
☒ Light
☐ Dark

Rotate Arr...Maximum...Get Bigge...DSA-with...C++ to Jav...LAPTOP-KR67QT5I...Sort Color...Remove D...Construct...Hackerblo...+New Chrome available

leetcode.com/problems/sort-colors/

logo bhैयाYouTubeMapsathiyarGmailAdobe AcrobatVideosAll Bookmarks

Problem ListSubmit1:1 Sessions with mePremium

DescriptionEditorialSolutionsSubmissions

75. Sort Colors

Solved

Discuss Approach

MediumTopicsCompaniesHint

Given an array `nums` with `n` objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers `0`, `1`, and `2` to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

Privacy and cookie settingsManaged by Google. Complies with IAB TCF. CMP ID: 300

21.5K6590 Online

Code

JavaAuto

```
1 class Solution {
2     public void sortColors(int[] nums) {
3
4     }
5 }
```

Help in coding > Saved

Test ResultTestcase

You must run your code first

(75. Sort Colors) → (counting approach)

(0...0...1...2)

→ [1 0 2 1 0 0]

(sort) 12 loops

Count Zero = 6 × 3

Count One = 6 × 2

Count Two = 6 × 1

Scanned with OKEN Scanner